

HACETTEPE ÜNİVERSİTESİ · YAZILIM MÜHENDİSLİĞİ  
YÜKSEK LİSANSI

# Nesneye Yönelik Yazılım Geliştirme

*Ara Sınav · Cevap Belgesi*

**Nurcan Denli Bayır**

Öğrenci No: N25110987

nurcandenli25@hacettepe.edu.tr

DÖNEM

2025-2026 Bahar

TESLİM TARİHİ

30 Nisan 2026

**B**u belge, sınavda istenen iki senaryo için nesneye yönelik tasarım çözümlerini sunar. Her soru için (i) senaryo çözümlemesi, (ii) tasarım örüntüsü seçimleri ile *reddedilmiş alternatiflerin gerekçeleri*, (iii) UML 2.x sınıf diyagramı, (iv) bir veya iki sıra (sequence) diyagramı, (v) önemli metotların sözde-kodları ve (vi) tipik çalışma akışları yer almaktadır. Diyagramlar, bağlılık (coupling) ve uyum (cohesion) ölçütleri ile *okunaklılık* dengesi gözetilerek elle dizayn edilmiş; otomatik olarak yerleştirilmemiştir.

## İÇİNDEKİLER

Soru 1 · Akıllı Ev Yönetim Sistemi ..... *State + Mediator + Observer*

Soru 2 · SimpleCar E-Satış Sistemi ..... *Strategy + CoR + Command*

## SORU 1 · 50 PUAN

# Akıllı Ev Yönetim Sistemi

State, Mediator ve Observer Örüntülerinin Birlikte Kullanımı

## 1.1 Senaryonun Çözümlemesi

Senaryo, dört tip bileşen tanımlar: iki *aktif sistem* (aydınlatma, ısıtma), iki *pasif ölçüm kaynağı* (kapı sensörü, hareket sensörü) ve bir *orta kontrol noktası* (akıllı ev izleme motoru). Bileşenler arasındaki etkileşimleri dört temel olguya indirgeyebiliriz:

- Davranışsal çoğulluk.** Aydınlatma ve ısıtma sistemleri, iç durumlarına — yani aktif *moda* — göre *farklı davranır*. Tasarruflu modda ısıtma 18°C, aydınlatma asgari seviye hedefler; güvenli modda ise sırasıyla 40°C panel sıcaklığı ve maksimum aydınlatma uygulanır. Bu davranışsal çoğulluğu **if/switch** ile çözmek, her yeni mod eklendiğinde mevcut sınıfı değiştirmeyi gerektirir. Polimorfizm üzerinden taşınması *açık-kapalı* prensibinin (OCP) doğal sonucudur.
- Koşullu senkronizasyon.** İki sistemin modu *bağlantılıdır*: bir sistemde tasarruflu mod seçilirse diğeri de zorunlu olarak geçer. Bu kural sistemler arasında *doğrudan* uygulanırsa, her sistem diğeri referans alır;  $n \times (n-1)$  bağlantısı ortaya çıkar. Senaryoda iki sistem var, ancak ileride yeni bir sistem (örn. sulama) eklendiğinde doğrudan referans yapısı kırılan hale gelir.
- Olay yayını, alıcı farkındalığı olmadan.** Sensörler, ölçümelerini iletmeli; ama *kime* ilettiklerini *bilmek zorunda olmamalı*. Sensörün motora doğrudan referansı olması, sensörü izleme altyapısına bağlar — sensörü başka bir bağlamda (örn. test, ayrı bir konsol) kullanmayı imkânsılaştırır.
- Çoğul bildirim hedefi.** Bir kullanıcının *birden fazla* istemcisi olabilir (mobil ve web). Motor, "kullanıcıya bildirim gönder" eylemini somut bir istemci tipine bağlarsa, yeni bir kanal (e-posta, SMS, sesli asistan) eklendiğinde motor değişir. Burada da *yayıncı-abone* ayrışması kaçınılmazdır.

## 1.2 Örüntü Seçimleri ve Gerekçeleri

Yukarıdaki dört olgu, GoF kataloğundaki üç *davranışsal* örüntü ile noktasal olarak eşleşir:

| ÖRÜNTÜ                            | UYGULANAN YER                                                                                                                                                              | ÇÖZDÜĞÜ OLGU                                                                                                                                                                     |
|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>State</b><br><i>GoF, s.305</i> | <code>CalismaModu</code> arayüzü; <code>GüvenliMod</code> ve <code>TasarrufluMod</code> ; <code>EvSistemi</code><br>«bağlam» rolünde, <code>aktifMod</code> alanını tutar. | (1) Davranışsal çoğulluk:<br><code>komutCalistir</code> çağırıldığında davranış aktif moda göre polimorfik seçilir. Yeni bir mod eklemek <code>EvSistemi</code> 'ni değiştirmez. |

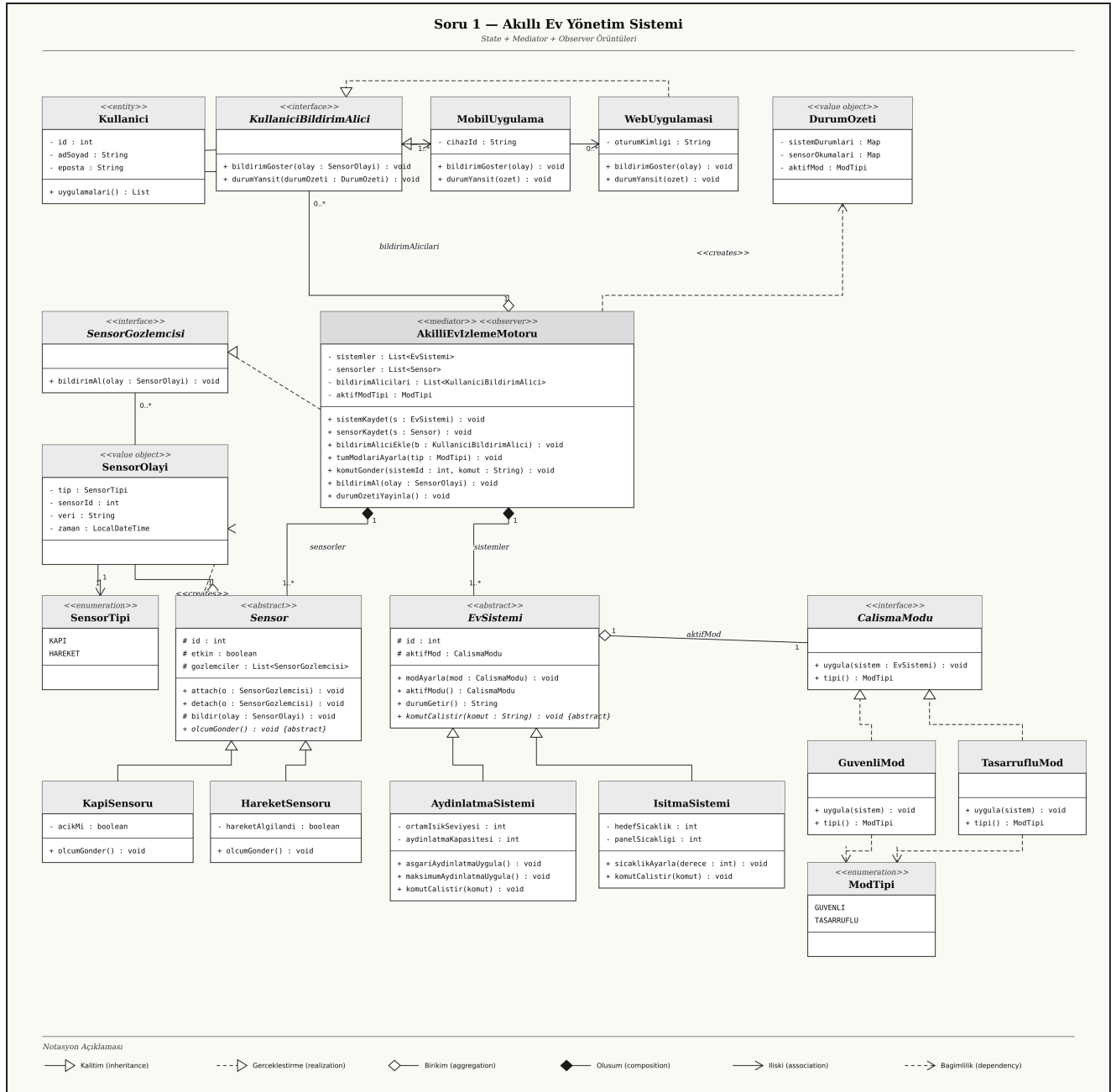
| ÖRÜNTÜ                               | UYGULANAN YER                                                                                                                                                          | ÇÖZDÜĞÜ OLGU                                                                                                                                                           |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mediator</b><br><i>GoF, s.273</i> | <b>AkıllıEvIzlemeMotoru</b> ; tüm <b>EvSistemi</b> ve <b>Sensor</b> nesnelerini tanır, sistemleri birbirine bağlamadan koordine eder.                                  | (2) Koşullu senkronizasyon: "tüm sistemler aynı moda geçsin" gibi çok-yönlü kurallar arabulucuda <i>tek bir noktada</i> uygulanır; sistemler arası referans gerekmez.  |
| <b>Observer</b><br><i>GoF, s.293</i> | (a) <b>Sensor</b> ↔ <b>SensorGozlemcisi</b> arayüzü (motor abonedir).<br>(b) <b>AkıllıEvIzlemeMotoru</b> ↔ <b>KullaniciBildirimAlici</b> arayüzü (mobil/web abonedir). | (3) Olay yayını ve (4) çoğul bildirim hedefi: hem sensör hem motor, abone tipinden bağımsız olarak yayın yapar. Yeni kanal/abone eklemek mevcut yayıncıyı değiştirmez. |

### 1.3 Reddedilen Alternatifler

| ALTERNATİF                                                         | NEDEN TERCİH EDİLMEDİ?                                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Singleton</b> — <b>AkıllıEvIzlemeMotoru</b> 'nu tekil yapmak.   | Senaryo "bir akıllı ev" den bahsediyor; ancak <i>test edilebilirlik</i> ve <i>gelecek çok-evli</i> (apartman, site) senaryosu için tekillik bağımlılığı maliyetli. Tekillik bir <i>üretim/yaratım</i> kararı; mevcut tasarım <i>davranışsal</i> sorunları çözer.       |
| <b>Strategy</b> — modları <b>CalismaStratejisi</b> olarak taşımak. | Strategy, davranışı <i>dışarıdan</i> seçilen bir algoritma olarak modeller; biz içine ait <i>durumu</i> modelliyoruz. Özellikle "iki sistemin modu birlikte değişir" kuralı, sistemden ayrı bir strateji nesnesini gerektirmez. State daha doğru kavramsal eşleşmedir. |
| <b>Pub/Sub mesaj kuyruğu</b> — sensör olayları için.               | Senaryo bunu istemez; ek altyapı maliyeti getirir. Observer, <i>aynı süreç içinde</i> ihtiyaç duyulan ayrışmayı sade biçimde sağlar.                                                                                                                                   |
| <b>Visitor</b> — mod değişikliğinde sistemleri dolaşmak için.      | Visitor, <i>sabit nesne hiyerarşisi üzerinde değişken işlemler</i> için uygundur. Burada işlem (mod değiştirme) sabit, hiyerarşi (sistemler) genişleyebilir; tam ters yön. Mediator + State daha uygun.                                                                |

**Mimari kanaat.** *State davranışı, Mediator koordinasyonu, Observer ise bilgi yayınına bağımsız eksenler olarak ayırır. Üç örüntü birbirinin kaldırıcı: birini çıkartmak diğer ikisinin yükünü artırır ve kodun açılan/kapanan dengesini bozar.*

### 1.4 Sınıf Diyagramı



Şekil 1.1 · Akıllı Ev Yönetim Sistemi — UML 2.x sınıf diyagramı. Sol katman: kullanıcı ve görüntüleme; orta katman: arabulucu motor ve gözlemci arayüzleri; alt katman: sistem ve sensör hiyerarşileri.

## 1.5 Sınıf Açıklamaları

| SINIF / ARAYÜZ         | STEREOTİP   | SORUMLULUK & ROLLER                                                                      |
|------------------------|-------------|------------------------------------------------------------------------------------------|
| Kullanici              | «entity»    | Ev sahibinin kimliği; bir veya birden fazla bildirim alıcıya (mobil/web) sahip olabilir. |
| KullaniciBildirimAlici | «interface» | Observer arayüzü (kanal-bağımsız).<br>bildirimGoster(olay) ve durumYansit(ozet) .        |

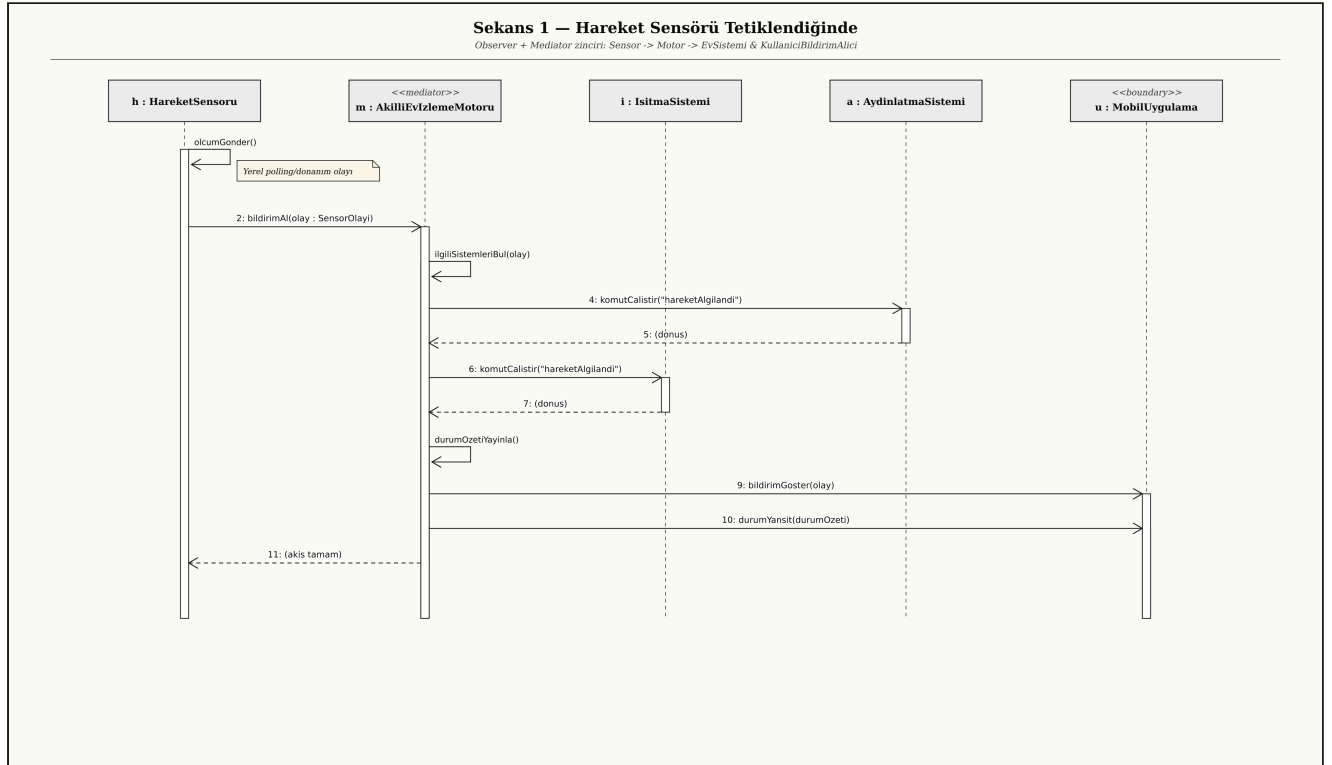
| SINIF / ARAYÜZ                       | STEREOTİP                  | SORUMLULUK & ROLLER                                                                                                                          |
|--------------------------------------|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| MobilUygulama .<br>WebUygulamasi     | «concrete»                 | Somut gözlemci; kanal-özel görüntüleme yapar. Kullanıcı komutlarını motora iletir.                                                           |
| DurumOzeti                           | «value object»             | Anlık durum fotoğrafı; değişmez (immutable). Motor üretir, alıcılar tüketir.                                                                 |
| AkıllıEvIzlemeMotoru                 | «mediator» +<br>«observer» | Tüm sistem ve sensörleri tanır. Sensör olaylarında gözlemci, kullanıcı için yayıncı. Mod senkronizasyonu, komut yönlendirme ve durum yayını. |
| EvSistemi                            | «abstract»                 | State için <i>bağlam</i> ; aktifMod alanı üzerinden davranışını polimorfik seçer. komutCalistir alt sınıflara bırakılır.                     |
| AydinlatmaSistemi .<br>IsitmaSistemi | «concrete»                 | Cihaz-özel davranış (asgari/maksimum aydınlatma; hedef sıcaklık) uygular.                                                                    |
| CalismaModu                          | «interface»                | State arayüzü; uygula(sistem) ile bağlamı yapılandırır.                                                                                      |
| GuvenliMod .<br>TasarrufluMod        | «concrete»                 | İki politik davranış: maksimum kapasite ile asgari/hedef ayar.                                                                               |
| ModTipi                              | «enumeration»              | Kullanıcı arayüzünden gelen mod tercihi için tip-güvenli sabitler.                                                                           |
| Sensor                               | «abstract»                 | Özne (subject) rolü: attach / detach ve bildir ortak metotları.                                                                              |
| KapiSensoru .<br>HareketSensoru      | «concrete»                 | Donanıma özgü ölçüm metotları ( olcumGonder ); olay tetiklerler.                                                                             |
| SensorGozlemcisi                     | «interface»                | Sensör olaylarını alacak nesneler için Observer arayüzü (motor uygular).                                                                     |
| SensorOlayi                          | «value object»             | Tip, kaynak sensör, veri ve zaman taşıyan değişmez olay nesnesi.                                                                             |
| SensorTipi                           | «enumeration»              | KAPI / HAREKET; mantık kontrolü ve kayıt için.                                                                                               |

## 1.6 İlişki Tipleri (UML Notasyonu)

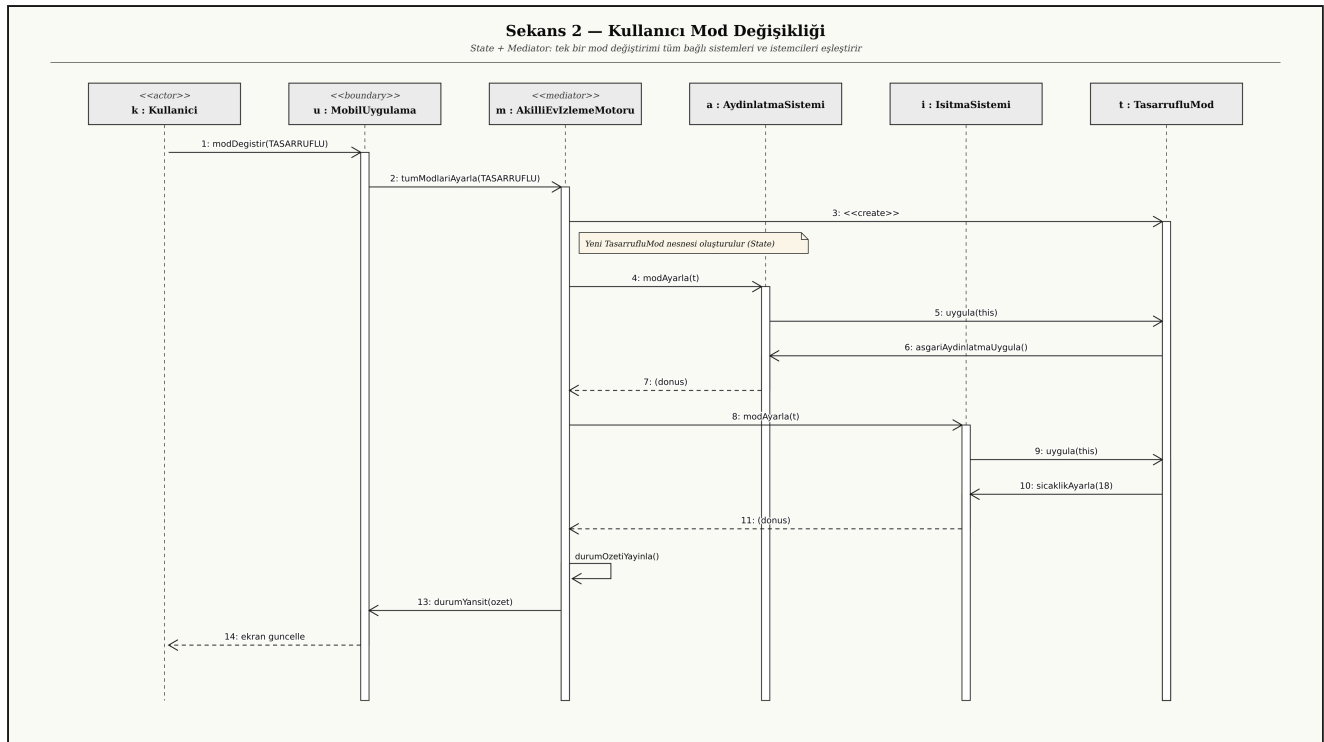
- Composition.* AkıllıEvIzlemeMotoru ◆— EvSistemi (1..\*) ve AkıllıEvIzlemeMotoru ◆— Sensor (1..\*) : motor olmadan sistem/sensör anlamlı değildir — yaşam döngülerini motor sahiplenir.

- *Aggregation.* AkıllıEvIzlemeMotoru ◊- KullaniciBildirimAlici (0..\*) : bildirim alıcıları motorun parçası değildir, dış dünyaya aittir; motor onları yalnızca tutar.
- *Aggregation.* EvSistemi ◊- CalismaModu (aktif mod): mod nesnesi sistemin yaşam döngüsünden bağımsız oluşturulup atanır.
- *Realization.* MobilUygulama / WebUygulamasi -..▷ KullaniciBildirimAlici ; GuvenliMod / TasarrufluMod -..▷ CalismaModu ; AkıllıEvIzlemeMotoru -..▷ SensorGozlemcisi .
- *Inheritance.* Sensor ve EvSistemi hiyerarşileri.
- *Dependency.* AkıllıEvIzlemeMotoru -..> DurumOzeti («creates»): motor durum özetini oluşturup yayınlr; alıcılar bunu yalnızca tüketir.

## 1.7 Sıra Diyagramları (Çalışma Akışları)



Şekil 1.2 · Hareket sensörü tetiklendiğinde olay akışı. Sensör olayı gözlemci üzerinden motora iletilir; motor ilgili sistemleri uyarır, sonra DurumOzeti ile tüm bildirim alıcılarını eşleştirir.



Şekil 1.3 · Kullanıcının tasarruflu mod talebi. Tek bir **tumModlariAyarla** çağırısı, mediator aracılığıyla tüm sistemleri ve istemcileri tutarlı biçimde günceller; sistemler birbirini referans almaz.

## 1.8 Önemli Metotların Sözde-Kodu

```
// MEDIATOR – tüm sistemleri tek noktadan moda geçir
class AkıllıEvIzlemeMotoru implements SensorGozlemcisi {
    void tumModlariAyarla(ModTipi tip) {
        CalismaModu yeniMod = (tip == ModTipi.TASARRUFLU)
                               ? new TasarrufluMod()
                               : new GuvenliMod();
        for (EvSistemi s : sistemler) {           // bağlantılı sistemler
            s.modAyarla(yeniMod);                 // aynı nesne paylaşılır
        }
        aktifModTipi = tip;
        durumOzetiYayinla();                      // observer'lara bildir
    }

    // OBSERVER – sensörden olay
    void bildirimAl(SensorOlayi olay) {
        for (EvSistemi s : ilgiliSistemleriBul(olay))
            s.komutCalistir(olay.tip == SensorTipi.HAREKET
                           ? "hareketAlgilandi" : "kapiAcildi");
        for (KullaniciBildirimAlici a : bildirimAlicilari)
            a.bildirimGoster(olay);               // kanal-bağımsız yayın
        durumOzetiYayinla();
    }

    private void durumOzetiYayinla() {
        DurumOzeti ozet = new DurumOzeti(/* anlık durum */);
        for (KullaniciBildirimAlici a : bildirimAlicilari)
            a.durumYansit(ozet);
    }
}
```



```
// STATE – davranışı mod üzerinden polimorfik taşı
abstract class EvSistemi {
    protected CalismaModu aktifMod;
    void modAyarla(CalismaModu m) { this.aktifMod = m; m.uygula(this); }
    abstract void komutCalistir(String komut);
}

class IsitmaSistemi extends EvSistemi {
    void sicaklikAyarla(int derece) { /* donanım çağrısı */ }
    void komutCalistir(String komut) {
        // mod kararının tüm yükü CalismaModu.uygula içinde toplanmıştır;
        // burada yalnızca komutu donanıma yansıtırız
    }
}

class TasarrufluMod implements CalismaModu {
    void uygula(EvSistemi s) {
        if (s instanceof IsitmaSistemi) ((IsitmaSistemi)
s).sicaklikAyarla(18);
        if (s instanceof AydinlatmaSistemi) ((AydinlatmaSistemi)
s).asgariAydinlatmaUygula();
    }
    ModTipi tipi() { return ModTipi.TASARRUFLU; }
}
```

## 1.9 Tasarımın Niteliksel Avantajları

- **Açık-Kapalı (OCP).** Yeni mod (UykuModu), yeni sensör (Pencere), yeni bildirim kanalı (E-posta); her durumda *yalnızca yeni bir sınıf* eklenir, mevcut kod değişmez.
- **Tek Sorumluluk (SRP).** Mod davranışı `CalismaModu` hiyerarşisinde; senkronizasyon politikası `AkıllıEvIzlemeMotoru` 'nda; olay taşıması `Sensor` + `SensorGozlemcisi` ikilisinde tutulmuştur.
- **Düşük Bağlılık.** Sistemler birbirini, sensörler de motoru somut bir tip olarak bilmez.
- **Test Edilebilirlik.** Stub gözlemci ve sahte modlar ile motor birim test seviyesinde doğrulanabilir.
- **Liskov Uyumu.** Yeni mod, yeni sistem ve yeni sensör türleri üst sözleşmenin (*contract*) ihlali olmadan eklenir.

## SORU 2 • 50 PUAN

# SimpleCar E-Satış Sistemi

Strategy, Chain of Responsibility ve Command Örüntülerinin Birlikte Kullanımı

## 2.1 Senaryonun Çözümlemesi

SimpleCar firması yalnızca e-satış yapan bir araba üreticisidir. Domeni dört bileşene ayrılabilir:

- 1. Algoritma ailesi (ödeme).** Ödeme tek bir *davranıştır*, ancak *banka havalesi*, *paypal*, *soğuk cüzdan* gibi farklı yollarla yapılır; senaryo ayrıca bu listenin *sıklıkla genişleyip daralacağını* özellikle vurgular. Çalışma zamanında seçilebilen, bağımsız olarak değiştirilebilen bir *algoritma ailesi* gerekir.
- 2. Aşamalı ve genişleyebilir kontrol.** Her sipariş *sahtecilik*, *limit* ve *bakiye* kontrollerine tabidir. Birinin olumsuzluğu siparişi iptal eder. Senaryo bu kontrollerin de zamanla *artırılabilirliğini* belirtir. Bu, bir *filtre zinciri*: her halka kararını verir, başarılıysa sıradakine devreder; başarısızsa zinciri keser.
- 3. Sıralı işleme adımları.** Tüm kontroller başarılıysa sipariş sırasıyla *fatura düzenleme*, *fatura gönderme* ve *alacaklara kaydetme* adımlarıyla işlenir. Adımlar ileride değişebilir; her biri test edilebilir bağımsız bir iş birimini temsil eder.
- 4. Yönetim ve yaşam döngüsü.** Bir orchestrator (use-case) bileşen, siparişi *alır*, *doğrular* ve *işler*. Domain (Siparis) ile servis sorumlulukları ayırık tutulmalıdır.

## 2.2 Örüntü Seçimleri ve Gerekçeleri

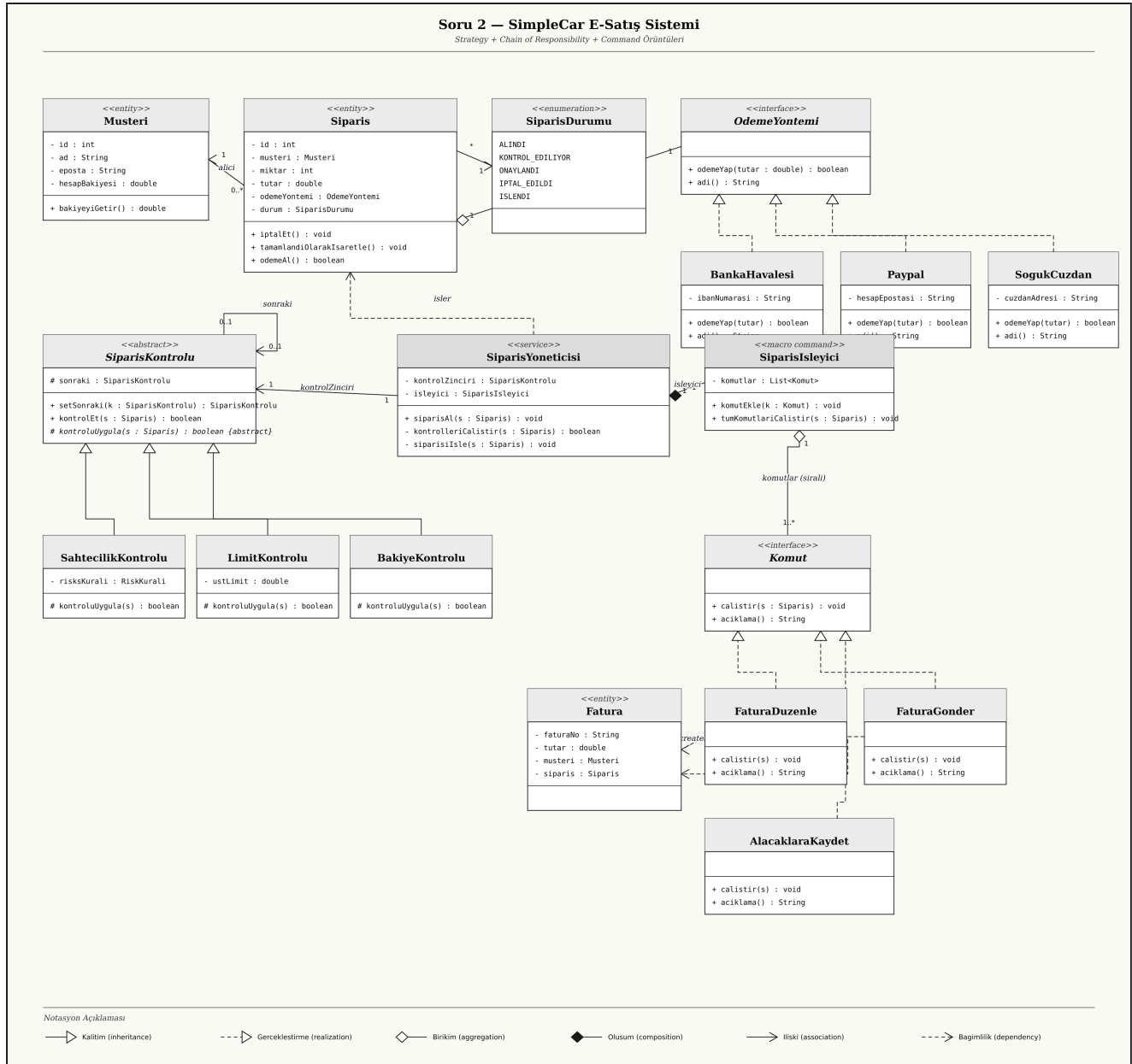
| ÖRÜNTÜ                                              | UYGULANAN YER                                                                                                                                                               | ÇÖZDÜĞÜ OLGU                                                                                                                             |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Strategy</b><br><i>GoF, s.315</i>                | <code>OdemeYontemi</code> arayüzü;<br><code>BankaHavalesi</code> , <code>Paypal</code> ,<br><code>SogukCuzdan</code> ; siparişle 1-1 birikim.                               | (1) Algoritma ailesi: ödeme ailesi sipariştten bağımsız olarak yer değiştirebilir; yeni yöntem eklemek mevcut kodu değiştirmez.          |
| <b>Chain of Responsibility</b><br><i>GoF, s.223</i> | <code>SiparisKontrolu</code> soyut sınıfı;<br><code>Sahtecilik</code> , <code>Limit</code> , <code>Bakiye</code><br>halkaları; <code>setSonraki</code> ile kuruluyor.       | (2) Aşamalı kontrol: halka başarılıysa sıradakine devreder, başarısızsa zincir kesilir; yeni kontrol eklemek tek satır konfigürasyondur. |
| <b>Command + Macro Command</b><br><i>GoF, s.233</i> | <code>Komut</code> arayüzü; <code>FaturaDuzenle</code> ,<br><code>FaturaGonder</code> , <code>AlacaklaraKaydet</code> ;<br><code>SiparisIsleyici</code> sırayla çalıştırır. | (3) Sıralı işleme adımları: her adım kapsüllenmiş; sıra değiştirme, ek adım, geri-alma (undo) gibi ihtiyaçlar için esnek altyapı.        |

## 2.3 Reddedilen Alternatifler

| ALTERNATİF                                                                          | NEDEN TERCİH EDİLMEDİ?                                                                                                                                                                                                            |
|-------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Template Method</b> — sipariş işleme akışını soyut sınıfta şablon olarak kurmak. | Şablon, <i>sıra sabit</i> ise iyidir; ancak senaryo "adım sayısı değişebilir" implikasyonunu taşır (yeni adımlar). Komut listesi, çalışma zamanında yeniden düzenlenebilir; şablon ise sınıfların yeniden yazılmasını gerektirir. |
| <b>Decorator</b> — kontrolleri sarmalayıcı olarak eklemek.                          | Decorator, ana arayüze yeni davranış ekler; ancak burada kontroller <i>kararsal</i> : false dönerse akışı durdurur. Bu, "ekleme" değil "eleme" semantiği; CoR daha doğru.                                                         |
| <b>Observer</b> — sipariş durumu değişikçe abonelere haber vermek.                  | Faydalı olabilirdi; ancak senaryo "bilgilendirme" istemiyor, <i>işlem akışını</i> tarif ediyor. Çözümümüz işlem-yönlüdür; Observer ihtiyaca göre daha sonra eklenebilir, mimariyi engellemez.                                     |
| <b>State</b> — sipariş durumu için.                                                 | <code>SiparisDurumu</code> bir enum yeterli; siparişin davranışı durumla değişmiyor (yalnızca basit gözlemlenebilir alanlar). State aşırı mühendislik olur.                                                                       |

**Mimari kanaat.** *Strategy "ne ile ödensin", CoR "kabul edilebilir mi", Command "kabul sonrası ne yapılsın" sorularını birbirinden bağımsız cevaplar. Senaryonun her üç değişme eksenini — ödeme yöntemleri, kontrol kuralları, işleme adımları — ayrı bir örüntü ile soyutlanmıştır; biri büyürken diğerleri sabit kalır.*

## 2.4 Sınıf Diyagramı



Şekil 2.1 · SimpleCar E-Satış — UML 2.x sınıf diyagramı. Sol kanat: CoR (kontroller); orta: orchestrator; sağ üst: Strategy (ödeme yöntemleri); sağ alt: Command (sıralı işleme adımları).

## 2.5 Sınıf Açıklamaları

| SINIF / ARAYÜZ | STEREOTİP     | SORUMLULUK & RÖLER                                                            |
|----------------|---------------|-------------------------------------------------------------------------------|
| Musteri        | «entity»      | Siparişi veren kişi; hesapBakiyesi bakiye kontrolünde kullanılır.             |
| Siparis        | «entity»      | Aggregate root; tutar, miktar, seçilen OdemeYontemi, durum bilgilerini taşır. |
| SiparisDurumu  | «enumeration» | Yaşam döngüsü sabitleri: ALINDI / KONTROL_EDILIYOR / ONAYLANDI /              |

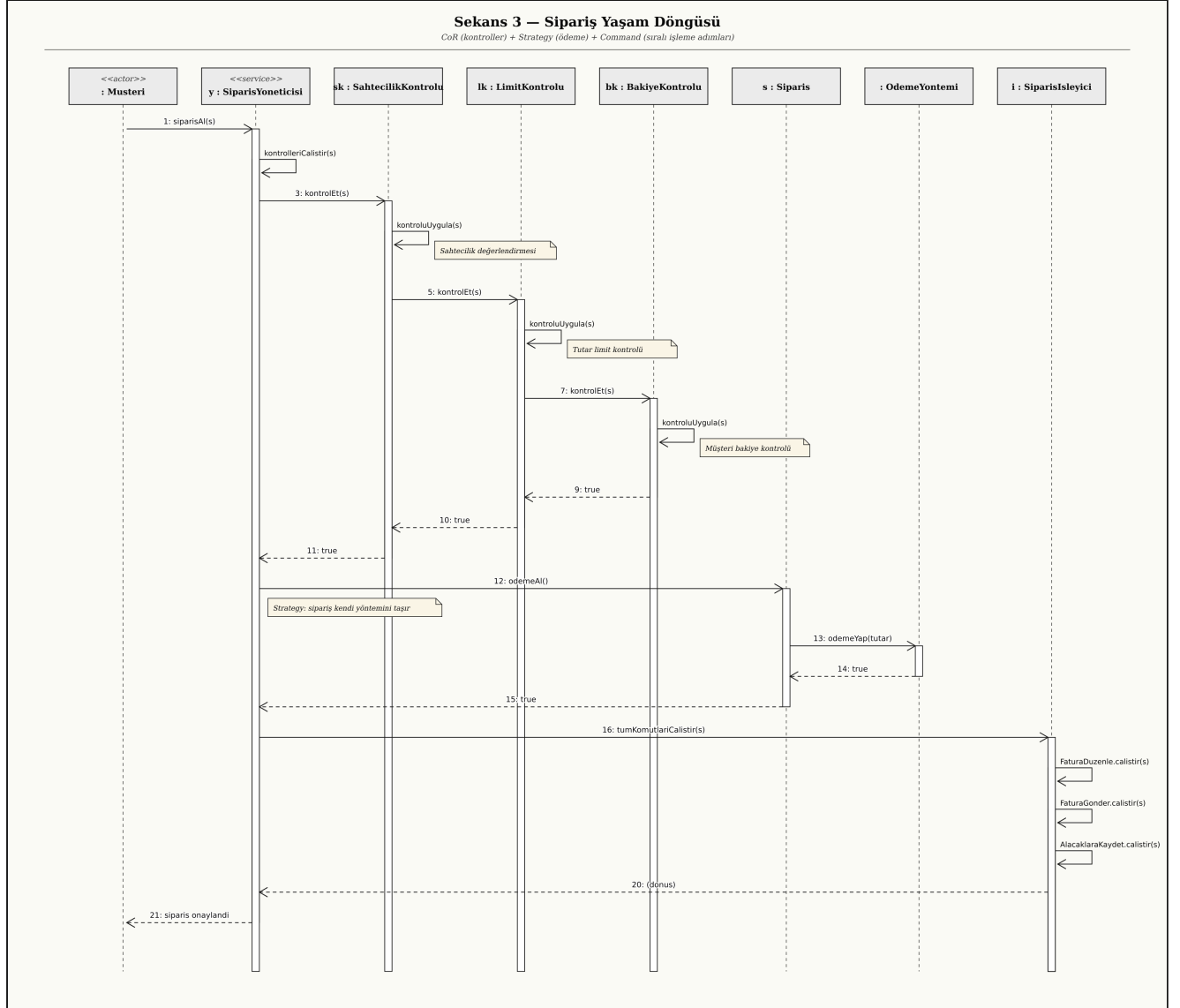
| SINIF / ARAYÜZ                                            | STEREOTİP       | SORUMLULUK & ROLLER                                                                                                                                                                     |
|-----------------------------------------------------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                           |                 | IPTAL_EDILDI / ISLENDI.                                                                                                                                                                 |
| OdemeYontemi                                              | «interface»     | Strategy sözleşmesi; <code>odemeYap(tutar)</code> imzası.                                                                                                                               |
| BankaHavalesi .<br>Paypal .<br>SogukCuzdan                | «concrete»      | Mevcut ödeme stratejileri; her birinin kanal-özel alanları (IBAN, e-posta, cüzdan adresi) vardır.                                                                                       |
| SiparisKontrolu                                           | «abstract»      | CoR taban sınıfı; <code>kontrolEt</code> şablon metodu zincirleme işini taşır, <code>kontrolUygula</code> alt sınıflara bırakılır.                                                      |
| SahtecilikKontrolu<br>· LimitKontrolu .<br>BakiyeKontrolu | «concrete»      | Tek bir kuralı uygulayan halkalar; bağımsız birim test yapılabilir.                                                                                                                     |
| Komut                                                     | «interface»     | Command sözleşmesi; <code>calistir(siparis)</code> .                                                                                                                                    |
| FaturaDuzenle .<br>FaturaGonder .<br>AlacaklaraKaydet     | «concrete»      | Senaryoda istenen sıralı üç işleme adımı; her biri kendi yan etkisinden sorumlu.                                                                                                        |
| SiparisIsleyici                                           | «macro command» | Sıralı komut listesi; <code>komutEkle</code> ile genişletilebilir.                                                                                                                      |
| SiparisYoneticisi                                         | «service»       | Use-case orchestratoru; kontrol zincirini ve işleyiciyi koordine eder. <i>Açık:</i> burada Facade kalıtsal bir izi vardır — çağırana tek bir <code>siparisAl(s)</code> arayüzü sunulur. |
| Fatura                                                    | «entity»        | Fatura komutları arasında paylaşılan veri; sipariş ile 1-1 ilişkili.                                                                                                                    |

## 2.6 İlişki Tipleri (UML Notasyonu)

- *Aggregation.* `Siparis` ◊— `OdemeYontemi` (1..1) : ödeme yöntemi siparişe atanır, fakat onun parçası değildir.
- *Aggregation.* `SiparisIsleyici` ◊— `Komut` (1..\*, sıralı) : komut listesi işleyiciden bağımsız olarak da var olabilir.
- *Composition.* `SiparisYoneticisi` ◆— `SiparisIsleyici` : yönetici, işleyiciyi kendisi oluşturur ve sahiplenir.
- *Self-association.* `SiparisKontrolu` — sonraki: 0..1 : zincirleme bağıntısı, halkanın kendisine.
- *Realization.* `BankaHavalesi/Paypal/SogukCuzdan` —..▷ `OdemeYontemi` ;  
`FaturaDuzenle/FaturaGonder/AlacaklaraKaydet` —..▷ `Komut` .
- *Inheritance.* Üç kontrol halkası `SiparisKontrolu` 'ndan türer.

- *Dependency*. SiparisYoneticisi –..> Siparis («invokes»); FaturaDuzenle –..> Fatura («creates»).

## 2.7 Sıra Diyagramı (Sipariş Yaşam Döngüsü)



Şekil 2.2 · Bir siparişin alınmasından ISLENDİ durumuna geçişine kadar tüm etkileşim akışı. Kontroller CoR boyunca devredilir; işleme adımları Macro Command tarafından sırayla çalıştırılır.

## 2.8 Önemli Metotların Sözde-Kodu

```
// CHAIN OF RESPONSIBILITY – taban sınıf
abstract class SiparisKontrolu {
    protected SiparisKontrolu sonraki;
    SiparisKontrolu setSonraki(SiparisKontrolu k) { this.sonraki = k; return k; }

    final boolean kontrolEt(Siparis s) {          // template method
        if (!kontrolUygula(s)) return false;    // başarısızsa zincir kesilir
        return sonraki == null ? true
            : sonraki.kontrolEt(s);
    }
    protected abstract boolean kontrolUygula(Siparis s);
}

class BakiyeKontrolu extends SiparisKontrolu {
    protected boolean kontrolUygula(Siparis s) {
        return s.musteri.bakiyeyiGetir() ≥ s.tutar;
    }
}
```

```
// MACRO COMMAND – sıralı komut listesi
class SiparisIsleyici {
    private final List<Komut> komutlar = new ArrayList<>();
    void komutEkle(Komut k) { komutlar.add(k); }
    void tumKomutlariCalistir(Siparis s) {
        for (Komut k : komutlar) k.calistir(s);
    }
}

// USE-CASE ORCHESTRATOR
class SiparisYoneticisi {
    private final SiparisKontrolu kontrolZinciri;
    private final SiparisIsleyici isleyici;

    void siparisAl(Siparis s) {
        s.durum = SiparisDurumu.KONTROL_EDILIYOR;
        if (!kontrolZinciri.kontrolEt(s)) {
            s iptalEt();                // durum = IPTAL_EDILDI
            return;
        }
        if (!s.odemeAl()) { s iptalEt(); return; }    // strategy üzerinden
        s.durum = SiparisDurumu.ONAYLANDI;
        isleyici.tumKomutlariCalistir(s);            // fatura düzenle/gönder/alacak
        s.tamamlandiOlarakIsaretle();                // durum = ISLENDI
    }
}
```

```
// STRATEGY – sipariştten seçilen yöntem çağırılır
class Siparis {
    private OdemeYontemi odemeYontemi;
    boolean odemeAl() { return odemeYontemi.odemeYap(this.tutar); }
}

class Paypal implements OdemeYontemi {
    private final String hesapEpostasi;
    public boolean odemeYap(double tutar) { /* ağ çağırısı */ return true; }
    public String adi() { return "Paypal"; }
}
```

## 2.9 Genişletilebilirlik Senaryoları

| YENİ GEREKSİNİM                                             | MEVCUT TASARIMDA YAPILACAK TEK ŞEY                                                                        |
|-------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Kripto kart yöntemi eklemek                                 | OdemeYontemi 'ni uygulayan yeni bir sınıf yazmak. Mevcut hiçbir kod değişmez.                             |
| KYC kontrolü eklemek                                        | SiparisKontrolu 'ndan türetilen yeni bir halka yazıp zincire setSonraki ile takmak.                       |
| Kargo etiketi oluşturma adımı eklemek                       | Komut 'u uygulayan yeni bir sınıfı SiparisIsleyici.komutEkle ile dahil etmek.                             |
| Bazı adımları paralel çalıştırma                            | SiparisIsleyici 'yi ParalelSiparisIsleyici olarak alternatifleştirmek (Liskov uyumlu).                    |
| Tüm kontrolleri tek seferde çalıştırıp tüm hataları toplama | Yeni bir HepsiniDogrulayanSiparisKontrolu kompozit halkası yazmak; mevcut halkalar değişmeden kullanılır. |

## 2.10 Sonuç

Önerilen tasarım; siparişin *alınması, doğrulanması ve işlenmesi* iç akışlarını birbirinden ayrı, örüntü-tabanlı kapsüllere yerleştirir. Senaryonun en sık değişeceği belirtilen iki eksen — *ödeme yöntemleri* ve *kontrol kuralları* — mevcut kodu değiştirmeden büyür. Sıralı işleme, gözlemlenebilir, sıra/komut listesi değiştirilebilir ve test edilebilir bir komut akışı ile gerçekleştirilir. Böylece, hem sınav kurallarında istenen *örüntü adı açıkça belirtilmiş* ve *ilişki tipi ile işaretlemeleri doğru kullanılmış*; hem de senaryodaki nitelikler ve metotlar sınıflar içinde konumlandırılmıştır.